

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : Damai Kerenhapukh Romainum

NIM : 109082530028

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

- Pengurutan (sorting)
- Selection Sort
- Insertion Sort

B. Guided

1. Sorting

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idxMin int

    for i := 0; i < len(arr)-1; i++ {
        idxMin = i

        for j := i + 1; j < len(arr); j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }

        if idxMin != i {
            arr[i], arr[idxMin] = arr[idxMin],
arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-
%d: ", i)
        fmt.Scan(&arrAngka[i])
    }

    fmt.Println("\n=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}
```

```

        SelectionSortArray(&arrAngka)

        fmt.Println("\n\n=== SETELAH SORTING ===")
        for i := 0; i < len(arrAngka); i++ {
            fmt.Print(arrAngka[i], " ")
        }
        fmt.Println()
    }
}

```

Output :

```

PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> go run
"c:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13\GUIDED\g
uided1\guidedno1.go"
Masukkan data angka indeks ke-0: 1
Masukkan data angka indeks ke-1: 3
Masukkan data angka indeks ke-2: 4
Masukkan data angka indeks ke-3: 5
Masukkan data angka indeks ke-4: 6
Masukkan data angka indeks ke-5: 2
Masukkan data angka indeks ke-6: 4
Masukkan data angka indeks ke-7: 6

=== SEBELUM SORTING ===
1 3 4 5 6 2 4 6

=== SETELAH SORTING ===
1 2 3 4 4 5 6 6
PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13>

```

2. sorting

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {

```

```

var idxMin int

for i := 0; i < len(sMHS)-1; i++ {
    idxMin = i

    for j := i + 1; j < len(sMHS); j++ {
        if sMHS[j].IPK < sMHS[idxMin].IPK {
            idxMin = j
        }
    }

    if idxMin != i {
        sMHS[i], sMHS[idxMin] = sMHS[idxMin], sMHS[i]
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n", i)

        fmt.Print("Nama: ")
        fmt.Scan(&structMHS[i].nama)

        fmt.Print("NIM: ")
        fmt.Scan(&structMHS[i].nim)

        fmt.Print("IPK: ")
        fmt.Scan(&structMHS[i].IPK)
    }

    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }

    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)

    fmt.Println("=== SETELAH SORTING ===")
}

```

```
        for i := 0; i < len(structMHS); i++ {  
            fmt.Printf("--- Data Mahasiswa ke-%d ---\n",  
i)                structMHS[i].nama)  
            fmt.Printf("Nama: %s\n", structMHS[i].nama)  
            fmt.Printf("NIM: %s\n", structMHS[i].nim)  
            fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)  
        }  
  
        fmt.Println("=====  
        fmt.Println()  
    }
```

Output :

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> go run "c:\Us
```

```
--- Masukkan Data Mahasiswa ke-0 ---
```

```
Nama: a
```

```
NIM: 1
```

```
IPK: 3.44
```

```
--- Masukkan Data Mahasiswa ke-1 ---
```

```
Nama: b
```

```
NIM: 2
```

```
IPK: 3.55
```

```
--- Masukkan Data Mahasiswa ke-2 ---
```

```
Nama: c
```

```
NIM: 3
```

```
IPK: 3.66
```

```
--- Masukkan Data Mahasiswa ke-3 ---
```

```
Nama: d
```

```
NIM: 4
```

```
IPK: 3.77
```

```
--- Masukkan Data Mahasiswa ke-4 ---
```

```
Nama: e
```

```
NIM: 5
```

```
IPK: 3.45
```

```
=== SEBELUM SORTING ===
```

```
--- Data Mahasiswa ke-0 ---
```

```
Nama: a
```

```
NIM: 1
```

```
IPK: 3.44
```

```
--- Data Mahasiswa ke-1 ---
```

```
Nama: b
```

```
NIM: 2
```

```
IPK: 3.55
```

```
--- Data Mahasiswa ke-2 ---
```

```
Nama: c
```

```
NIM: 3
```

```
IPK: 3.66
```

```
--- Data Mahasiswa ke-3 ---
```

```
Nama: d
```

```
NIM: 4
```

```
IPK: 3.77
```

```

--- Data Mahasiswa ke-3 ---
Nama: d
NIM: 4
IPK: 3.77
--- Data Mahasiswa ke-4 ---
Nama: e
NIM: 5
IPK: 3.45
=====

=== SETELAH SORTING ===
--- Data Mahasiswa ke-0 ---
Nama: a
NIM: 1
IPK: 3.44
--- Data Mahasiswa ke-1 ---
Nama: e
NIM: 5
IPK: 3.45
--- Data Mahasiswa ke-2 ---
Nama: b
NIM: 2
IPK: 3.55
--- Data Mahasiswa ke-3 ---
Nama: c
NIM: 3
IPK: 3.66
--- Data Mahasiswa ke-4 ---
Nama: d
NIM: 4
IPK: 3.77
=====

```

3. Sorting

```

package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int

    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j--
        }
    }
}

```

```

        }

        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data:")
    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("\nData sebelum sorting:")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting
(Descending):")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}

```

Output :


```
PROBLEMS  TERMINAL  ...  Code + -  [ ]  [ ]  ...  |  [ ]  X

PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> go run
"c:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13\GUIDED\g
uided3\guidedno3.go"
Masukkan jumlah data: 3
Masukkan data:
16 2 7

Data sebelum sorting:
16 2 7

Data setelah sorting (Descending):
16 7 2
PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> |
```

4. Sorting

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   string
    ipk   float64
}

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
```

```

        j = i - 1

        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j--
        }

        T[j+1] = temp
    }
}

func main() {
    var data arrMhs
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData mahasiswa ke-", i+1)

        fmt.Print("Nama: ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM: ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK: ")
        fmt.Scan(&data[i].ipk)
    }
}

```

```
}

fmt.Println("\nData sebelum sorting:")
for i = 0; i < n; i++ {
    fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
}

insertionSortStruct(&data, n)

fmt.Println("\nData setelah sorting (Descending berdasarkan
Nama):")
for i = 0; i < n; i++ {
    fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
}
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> go run "c:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13>
Masukkan jumlah mahasiswa: 3

Data mahasiswa ke- 1
Nama: damai
NIM: 1
IPK: 3.44

Data mahasiswa ke- 2
Nama: berto
NIM: 2
IPK: 3.11

Data mahasiswa ke- 3
Nama: dirga
NIM: 3
IPK: 3.56

Data sebelum sorting:
damai 1 3.44
berto 2 3.11
dirga 3 3.56

Data setelah sorting (Descending berdasarkan Nama):
dirga 3 3.56
damai 1 3.44
berto 2 3.11
PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> |
```

C. Unguided

1. Rumah kerabat

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }

        temp := arr[idxMin]
        arr[idxMin] = arr[i]
        arr[i] = temp
    }
}
```

```

    }
}

func main() {
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var rumah []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            rumah = append(rumah, nomor)
        }

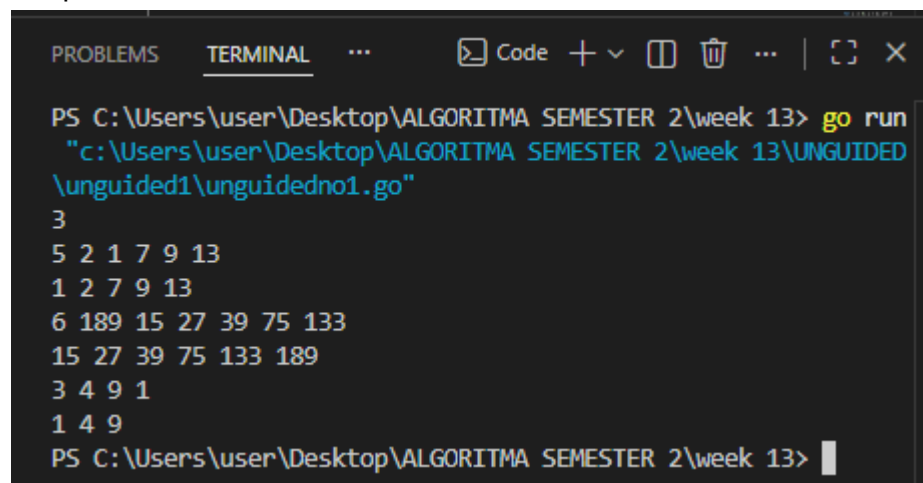
        selectionSort(rumah, m)

        for j := 0; j < m; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(rumah[j])
        }

        fmt.Println()
    }
}

```

Output :



```

PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> go run
"c:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13\UNGUIDED
\unguided1\unguidedno1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13>

```

Penjelasan :

Program ini digunakan untuk mengurutkan nomor rumah menggunakan algoritma Selection Sort secara ascending (dari kecil ke besar). Pengguna memasukkan beberapa kelompok data, kemudian setiap nomor rumah disimpan ke dalam slice dan diurutkan dengan mencari nilai terkecil pada setiap iterasi lalu menukarnya ke posisi yang sesuai. Setelah proses pengurutan selesai, program menampilkan nomor rumah yang telah tersusun secara berurutan untuk setiap kelompok data.

2. Kerabat Dekat

- 2) Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
    }
}
```

```

    }

    temp := arr[idxMin]
    arr[idxMin] = arr[i]
    arr[i] = temp
}

func selectionSortDesc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMax := i

        for j := i + 1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }

        temp := arr[idxMax]
        arr[idxMax] = arr[i]
        arr[i] = temp
    }
}

func main() {
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)

            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }

        selectionSortAsc(ganjil, len(ganjil))
        selectionSortDesc(genap, len(genap))

        first := true
    }
}

```



```

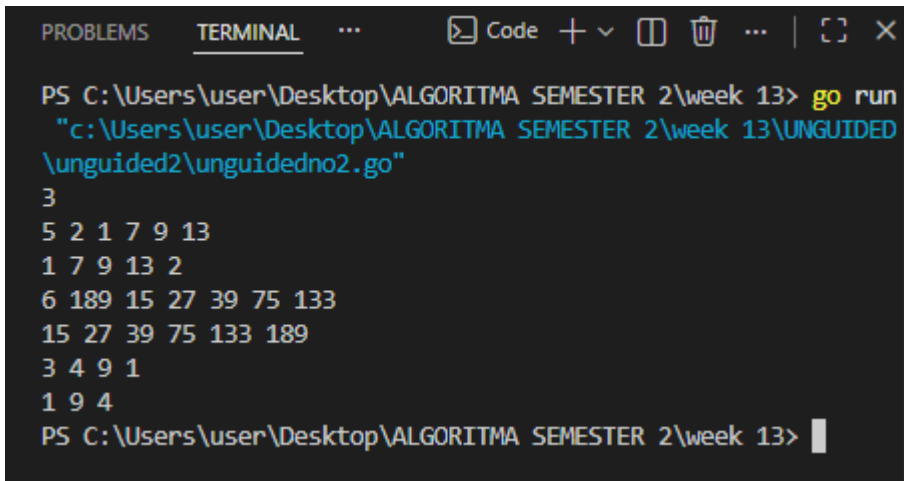
        for j := 0; j < len(ganjil); j++ {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(ganjil[j])
            first = false
        }

        for j := 0; j < len(genap); j++ {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(genap[j])
            first = false
        }

        fmt.Println()
    }
}

```

Output :



```

PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> go run
"c:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13\UNGUIDED
\unguided2\unguidedno2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13>

```

Penjelasan :

Program ini digunakan untuk mengelompokkan bilangan menjadi bilangan ganjil dan genap, kemudian mengurutkannya menggunakan algoritma Selection Sort. Bilangan ganjil diurutkan secara ascending (dari kecil ke besar), sedangkan bilangan genap diurutkan secara descending (dari besar ke kecil). Setelah proses pengurutan selesai, program menampilkan seluruh bilangan ganjil terlebih dahulu, kemudian diikuti bilangan genap yang telah diurutkan sesuai ketentuan. Dengan demikian, hasil akhir menampilkan data yang sudah terpisah berdasarkan jenis bilangan dan tersusun secara teratur.

3. Jarak

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 1 7 19 37 -5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main

import "fmt"

func insertionSort(arr []int, n int) {
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i

        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j--
        }

        arr[j] = temp
    }
}

func main() {
    var arr []int
    var input int

    for {
        fmt.Scan(&input)
```

```

        if input < 0 {
            break
        }

        arr = append(arr, input)
    }

    n := len(arr)

    if n > 0 {
        insertionSort(arr, n)
    }

    for i := 0; i < n; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(arr[i])
    }

    fmt.Println()

    if n < 2 {
        fmt.Println("Data berjarak tidak tetap")
    } else {
        jarak := arr[1] - arr[0]
        tetap := true

        for i := 1; i < n-1; i++ {
            if arr[i+1]-arr[i] != jarak {
                tetap = false
                break
            }
        }

        if tetap {
            jarak)
            fmt.Printf("Data berjarak %d\n",
            tetap")
        } else {
            fmt.Println("Data berjarak tidak
            tetap")
        }
    }
}

```

Output :

```
PROBLEMS  TERMINAL  ...  Code  +  -  [ ]  [X]  ...  [ ]  X

PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> go run
"c:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13\UNGUIDED
\unguided3\unguided3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Users\user\Desktop\ALGORITMA SEMESTER 2\week 13> |
```

Penjelasan :

Program ini digunakan untuk membaca sejumlah bilangan hingga pengguna memasukkan bilangan negatif sebagai tanda berhenti. Seluruh data yang telah dimasukkan kemudian diurutkan secara ascending menggunakan algoritma Insertion Sort. Setelah data terurut, program memeriksa apakah selisih antar bilangan berurutan selalu sama. Jika selisihnya tetap, program menampilkan besar jaraknya. Namun, jika terdapat perbedaan selisih antar data, program akan menampilkan bahwa data memiliki jarak yang tidak tetap.

4. XX

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax ] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Jawaban :

```
package main

import "fmt"

const nMax int = 7919
```

```

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit,
            &pustaka[i].eksemplar,
            &pustaka[i].tahun,
            &pustaka[i].rating,
        )
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }

    maxRating := pustaka[0].rating
    idx := 0

    for i := 1; i < n; i++ {
        if pustaka[i].rating > maxRating {
            maxRating = pustaka[i].rating
            idx = i
        }
    }

    fmt.Printf(
        "%s %s %s %d\n",
        pustaka[idx].judul,
        pustaka[idx].penulis,
        pustaka[idx].penerbit,
        pustaka[idx].tahun,
    )
}

```

```

    )
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := pustaka[i]
        j := i

        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j--
        }

        pustaka[j] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5

    if n < 5 {
        limit = n
    }

    for i := 0; i < limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    kiri := 0
    kanan := n - 1
    found := false
    idx := -1

    for kiri <= kanan && !found {
        tengah := (kiri + kanan) / 2

        if pustaka[tengah].rating == r {
            found = true
            idx = tengah
        } else if r > pustaka[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }
}

```

```

        }
    }

    if found {
        fmt.Printf(
            "%s %s %s %d %d %d\n",
            pustaka[idx].judul,
            pustaka[idx].penulis,
            pustaka[idx].penerbit,
            pustaka[idx].tahun,
            pustaka[idx].eksemplar,
            pustaka[idx].rating,
        )
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti
itu")
    }
}

func main() {
    var pustaka DaftarBuku
    var n, r int

    DaftarkanBuku(&pustaka, &n)

    CetakTerfavorit(pustaka, n)

    UrutBuku(&pustaka, n)

    Cetak5Terbaru(pustaka, n)

    fmt.Scan(&r)

    CariBuku(pustaka, n, r)
}

```

Output :

```

6
B01 Algoritma_Dasar Budi Informatika_Press 10 2020 85
B02 Pemrograman_Go Andi Telkom_Press 5 2021 95
B03 Struktur_Data Citra IT_Books 15 2019 70
B04 Basis_Data Doni SQL_Pub 8 2022 88
B05 Jaringan_Komp Eka Net_Press 12 2018 75
B06 AI_Lanjut Fajar Future_Books 20 2023 92
Pemrograman_Go Andi Telkom_Press 2021
Pemrograman_Go
AI_Lanjut
Basis_Data
Algoritma_Dasar
Jaringan_Komp
88
Basis_Data Doni SQL_Pub 2022 B 88

```

Penjelasan :

Program ini digunakan untuk mengelola data buku dalam sebuah perpustakaan. Pengguna memasukkan sejumlah data buku yang berisi ID, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Program kemudian mencari dan menampilkan buku dengan rating tertinggi sebagai buku terfavorit. Setelah itu, data buku diurutkan berdasarkan rating secara menurun menggunakan algoritma Insertion Sort, lalu menampilkan maksimal lima judul buku dengan rating tertinggi. Terakhir, program melakukan pencarian buku berdasarkan rating tertentu menggunakan Binary Search dan menampilkan informasi lengkap buku jika ditemukan, atau pesan bahwa buku tidak ditemukan jika rating yang dicari tidak ada.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting dan searching sangat penting dalam pengolahan data. Algoritma seperti Selection Sort dan Insertion Sort digunakan untuk mengurutkan data agar lebih terstruktur, sedangkan Sequential Search dan Binary Search digunakan untuk menemukan data dengan lebih mudah dan cepat. Melalui implementasi menggunakan bahasa Go, praktikum ini membantu memahami cara kerja algoritma dasar serta penerapannya dalam berbagai kasus pengolahan data.

